

1 Introduction

This document aims to explain the work done about the Automated Reasoning project assignment.

The project consists in coding a program that solves a Constraint Optimization Problem (COP).

The program must be coded both in Minizinc and in ASP (Answer Set Programming).

For Minizinc I used the Minizinc IDE 2.8.5.

For ASP we used VScode to code and Clingo 5.7.1 to compile and execute.

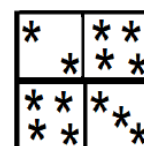
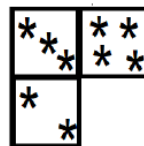
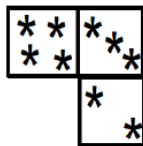
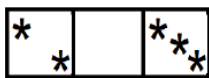
2 Problem Statement

The problem in analysis requires to work on a version like that of the famous domino game.

Suppose you have several tiles from an advanced version of the domino and a square playing table represented by a $n * n$ matrix.

The tiles have various forms:

- **Linear** with 3 cells
- **L-shaped** with 3 cells
- **Squared** $2 * 2$



Parameters like:

- Matrix Dimension
- Number of input Tiles
- Number of cells of each tile
- Values inside cells of tiles (the single value could be any number between 0 and 6)

are given in **input**.

The problem is to **place** the **tiles** in the **game board**, **maximizing** the **number** of **tiles** we can put down according to this rule: **All tiles must be connected**.

When two **tiles** are **adjacent**, we must have the **same number** within the neighboring cells.

Of course, there can be more than one side matching.

So each side in contact must have the same number, even if there are multiple points of contact with cells of other tiles.

3 Minizinc

This section examines the Minizinc code part.

Additional comments can be found in the source file.

3.1 Input

n -----> Grid dimension (n x n)

tile_number -----> Total number of Tiles in input

number_of_cells -----> Array of how many cell there are in each tile (one number for each tile)

tiles -----> Two-dimensional Array of Tuple

Input simple example:

```
easy_input_3.dzn 
1 n = 4;
2
3 num_tessere = 5;
4
5 numero_di_celle = [3,3,3,3,4];
6
7 tessere = [| (1, 0, 0), (2, 0, 1), (3, 1, 0), (0, 0, 0) | % fuxia
8             (1, 0, 0), (2, 0, 1), (3, 1, 0), (0, 0, 0) | % fuxia
9             (1, 0, 0), (2, 1, 0), (3, 1, 1), (0, 0, 0) | % arancione
10            (1, 0, 0), (2, 1, 0), (3, 1, 1), (0, 0, 0) | % arancione
11            (1, 0, 0), (2, 0, 1), (3, 1, 0), (4, 1, 1) |]; % viola
12
```

Note :

- Each tiles row represents a different tile
 - The first tile has row index 1 (so become the **tile number 1**)
 - The last tile has row index tile_number (so become the **tile number tile_number**)
- Each tuple is composed by 3 integer numbers
 1. cell value (between 0 and 6)
 2. cell coordinate on the cartesian axe X (in an ideal 3 * 3 matrix)
 3. cell coordinate on the cartesian axe Y (in an ideal 3 * 3 matrix)

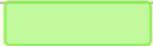
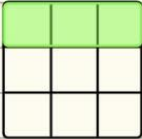
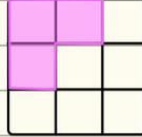
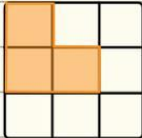
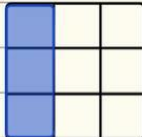
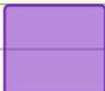
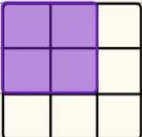
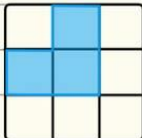
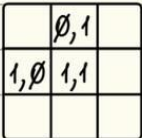
Each row of the “tiles” two-dimensional array is composed by 4 tuples.

The last one of these is set to (0,0,0) to show that the tile is composed of only 3 cells.

To have a better look of what are the tiles in the input files, I've assigned **colors** to identify the **shape** of a tile:

1. Green
2. Fuxia
3. Orange
4. Blue
5. Purple
6. Light Blue

This has no counterpart in the code but facilitate a human eye analysis.

1			
2			
3			
4			
5			
6			

3.2 Useful information

The code uses these **decisional variables** :

- **grid** : represent the board table; when a tile is placed on the game board, this $n \times n$ matrix will contain the cell values of the placed tiles
- **place_grid** : represent the board table; when a tile is placed on the game board, this $n \times n$ matrix will contain the index values of the placed tiles
- **placed_tile** : binary array that associates each position with the respective tile number (0 for a not placed tile, 1 for a placed tile).
The code tries to maximize the sum this array elements.
- **connected** : matrix of binary variables indicating whether a cell is connected to the main component

3.3 Constraints

Here is a brief explanation of each constraint used.

Inside the provided Minizinc code it is possible to find comments and numeration of the various constraints.

1) Place tiles in grid and place_grid

This constraint allows to assess "whether a tile can be placed in a given position".

There must be a **valid starting position**, identified by X and Y coordinates, such that the conditions indicated below are met. For this purpose, an auxiliary constant "k" is used to track the number of cells belonging to the tile under analysis.

It implements a cycle that runs along each tile cell and new coordinates are obtained to be analyzed, identified by "cellX" and "cellY".

They are obtained by adding the **coordinates** of the **starting position** and an **offset** relative to the **coordinates** of the **cell under analysis**.

For each of them, it is required that the value in "**grid**" corresponds to the value that the cell must contain (provided as input) and that the value in "**place_grid**" corresponds to that of the tile under analysis.

It is also ensured that the tiles are placed within the **limits of the playing board** and there can be **no overlapping** of several cells in the same position.

If conditions are met for all cells of the tile, it is possible to place the tile on the board.

2) Each tile must be adjacent to at least one other tile

This constraint is valid for all tiles outside of the first one that is placed on the game board.

Ensures that a tile can be placed if there is a cell, belong to it, which is adjacent to a cell of a different tile.

3) Adjacent cells but of different tiles must have the same value

This constraint requires that two cells of different adjacent tiles have the same value within them. It is applied to each cell of the "**place_grid**" matrix in which a tile has been placed.

To perform the check, a **starting cell** is used and the **four adjacent cells** to it are analysed.

4) Limit of usable cells to the number of cells in input tiles

This constraint requires that each tile occupies a number of cells in the "**place_grid**" matrix corresponding to the number of cells it is composed of.

5,6) A cell is occupied if it has a tile placed, start to connect from there

These constraints require that each cell of the "connected" matrix is set to 1 if in the "**place_grid**" matrix there is a value other than 0.

In other words, if you have placed a tile cell at those certain coordinates of the game board.

7) Propagation of the connection in all directions to occupied adjacent cells

This constraint allows, if a cell of the game board is occupied, to propagate the connection to the main component to all adjacent components.

The objective is to ensure that all tiles placed are part of a single connected component, **preventing isolated groups of tiles**.

3.4 Output

The output shows :

- The "**Game Board**" grid with the corresponding **tile cell value** for each cell
- The "**Index Tiles placing Board**" grid with the corresponding **tile number** for each cell
- Array with indexes of **correctly placed tiles**

Disclaimer :

Note that 0 is a valid value that can be in a cell, but also the default value for unused cells in "grid".

To avoid confusion when displaying the output, note that a 0 in "**grid**" indicates that a cell is empty only if, at the same coordinates, there is also a 0 in "**place_grid**".

3.5 Test Results

Input files can be found inside the minizinc folder.

You can find 2 versions of the program: one Italian and one English.

Choose the one you prefer, note to use the corresponding input for that language.

Input files are classified as:

- Easy difficulty (5 files)
- Medium difficulty (2 files)
- Hard difficulty (3 files)

Maximizable express if all the input tiles can be correctly placed on the grid.
If no, the program will maximize the number of placed tiles.

Compl. Time express the time required to place all tiles that can be placed.

These are the time results of the test:

Input Name	Solved?	Time	Placed tiles	Maximizable?	Compl. time
Easy 1	✓	500 ms	2/2	✓	
Easy 2	✓	3,2 seconds	3/4	✗	
Easy 3	✓	4,1 seconds	2/5	✗	
Easy 4	✓	5,3 seconds	4/4	✓	
Easy 5	✓	2,7 seconds	4/4	✓	
Medium 1	✓	2,2 minutes	4/4	✓	
Medium 2	✓	3,1 minutes	4/4	✓	
Hard 1	Partially	Timeout	4/5	✓	6 minutes
Hard 2	Partially	Timeout	4/5	✓	12 minutes
Hard 3	Partially	Timeout	4/5	✓	+50 minutes

If we take a closer look at the **Hard_1** input file, we see that at timeout time the solution found is the one on the left.

But it could be possible to include also the linear (blue) tile as showed on the right.

To do this, is required more then the 5 minutes timeout time, as showed below.

1	3					1	1	3			
2	4	4	2			2	2	4	4	2	
		3	4			3			3	4	
1	2	3					1	2	3		
1	2	3					1	2	3		
4/5						5/5					

Griglia:

```
[| 1, 1, 3, 0, 0
 | 2, 2, 4, 4, 2
 | 3, 0, 0, 3, 4
 | 0, 1, 2, 3, 0
 | 0, 1, 2, 3, 0
 |]
```

Posizionamento delle tessere:

```
[| 3, 5, 5, 0, 0
 | 3, 5, 5, 4, 4
 | 3, 0, 0, 4, 4
 | 0, 1, 1, 1, 0
 | 0, 2, 2, 2, 0
 |]
```

Tessere posizionate: [1, 2, 3, 4, 5]

=====

Finished in 6m 48s.

4 Answer Set Programming

4.1 Input

The tiles in input to the ASP program are the same used for the Minizinc code.

You can find them in the asp folder.

All the input are a series of facts, we define as done in Minizinc the dimension of the grid, the number of input tiles and the input tiles themselves.

Each tile is expressed in this way :

```
tile(1, 1, 1, 0, 0).  
tile(1, 2, 2, 0, 1).  
tile(1, 3, 3, 0, 2).  
  
tile(2, 1, 1, 0, 0).  
tile(2, 2, 2, 0, 1).  
tile(2, 3, 3, 1, 0).
```

We use a number of facts for each tile that is equal to the number of cell that the tile is made from.

In this example we have 3 facts that define the tile 1 and 3 facts that define the tile 2.

Each one of these facts has 5 parameters :

1. Tile Identifier
2. Number of cell that belongs to the tile
3. Value that is associated with the cell
4. X offset coordinate
5. Y offset coordinate

The 2 offset coordinates are used to obtain the right shape of the tile in an hypothetical 3*3 grid.

Following this idea, the tile 1 is linear, horizontal and made of 3 cells with values: 1 for the first cell, 2 for the second cell, 3 for the third cell.

4.2 Useful information

This program finds in the optimal solution the tile combination on the grid that maximizes the number of used tiles.

Note that the requirement “tiles on the grid must be all connected” was difficult to satisfy because there can be 2 groups or more groups of different adjacent tiles.

So, the program tries to find a solution such that tiles form a connected group.

4.3 Constraints

Comment about can be found in the code.

- Definition of the adjacency predicate to identify if two cells of the grid are close to each other.
- Definition of the game grid, composed by X and Y coordinates between 1 and the dimension of the grid (found in the input file)
- The solver decides which tiles to utilize by means of a decisional variable `use_tile(.)`
- Assign exactly 1 base position on the grid for each tile.
This position is the starting point, I will add to its coordinates the offset ones relative to the tile.
- The solver can decide if place a cell of the tile on the gaming grid.
- Make sure that if a tile is not used, none of its cells should be positioned
- If a tile is used, all its cells must be placed
- We consider a tile as connected if has a cell that is adjacent to a cell of another tile
- Search if there is a direct link between two tiles on the grid
- Then connect tiles that are not directly adjacent but part of the same chain
- No cell can be used by more than one tile
- Two cells of different tiles can be adjacent only if they have the same value
- Every tile used must be connected (except if it is the only one)
- If we use more than one tile, they must all be reachable from the first used tile

4.4 Output

The output of the ASP program is given as a set of atom:

```
use_tile(1) use_tile(2) positioned(1,1,1,1) positioned(2,1,2,1) positioned(2,3,3,1) positioned(1,2,1,2)
positioned(2,2,2,2) positioned(1,3,1,3)
Optimization: -2
OPTIMUM FOUND
```

- **use_tile** is referred to which of the tiles was used
- **positioned** has 4 predicates :
 - 1) The number of the referred tile
 - 2) The number of the cell of the referred tile
 - 3) X coordinate on the game grid of where the cell is placed
 - 4) Y coordinate on the game grid of where the cell is placed

The optimization value shows how much tiles are used in the provided solution.

4.5 Tests

These are the results of the test done on the ASP program, with a time limit of 5 minutes to execute.

Note that in the next page, the red number inside cells of tiles is the respective number that is there contained.

Input Name	Solved?	Time	Placed tiles	Maximizable?
Easy 1	✓	30 ms	2/2	✓
Easy 2	✓	40 ms	3/4	✗
Easy 3	✓	40 ms	2/5	✗
Easy 4	✓	40 ms	4/4	✓
Easy 5	✓	50 ms	4/4	✓
Medium 1	✓	50 ms	4/4	✓
Medium 2	✓	45 ms	4/4	✓
Hard 1	✓	65 ms	5/5	✓
Hard 2	✓	50 ms	5/5	✓
Hard 3	✓	60 ms	5/5	✓

Solution found for each input file.

EASY_1.lp

1,1	1,2	1,3	1,4
2,1	2,2	2,3	2,4
3,1	3,2	3,3	3,4
4,1	4,2	4,3	4,4

EASY_2.lp

1,1	1,2	1,3	1,4
2,1	2,2	2,3	2,4
3,1	3,2	3,3	3,4
4,1	4,2	4,3	4,4

EASY_3.lp

1,1	1,2	1,3	1,4
2,1	2,2	2,3	2,4
3,1	3,2	3,3	3,4
4,1	4,2	4,3	4,4

EASY_4.lp

1,1	1,2	1,3	1,4	1,5
2,1	2,2	2,3	2,4	2,5
3,1	3,2	3,3	3,4	3,5
4,1	4,2	4,3	4,4	4,5
5,1	5,2	5,3	5,4	5,5

EASY_5.lp

1,1	1,2	1,3	1,4	1,5
2,1	2,2	2,3	2,4	2,5
3,1	3,2	3,3	3,4	3,5
4,1	4,2	4,3	4,4	4,5
5,1	5,2	5,3	5,4	5,5

MEDIUM_1.lp

1,1	1,2	1,3	1,4	1,5
2,1	2,2	2,3	2,4	2,5
3,1	3,2	3,3	3,4	3,5
4,1	4,2	4,3	4,4	4,5
5,1	5,2	5,3	5,4	5,5

MEDIUM_2.lp

1,1	1,2	1,3	1,4	1,5
2,1	2,2	2,3	2,4	2,5
3,1	3,2	3,3	3,4	3,5
4,1	4,2	4,3	4,4	4,5
5,1	5,2	5,3	5,4	5,5

HARD_1.lp

1,1	1,2	1,3	1,4	1,5
2,1	2,2	2,3	2,4	2,5
3,1	3,2	3,3	3,4	3,5
4,1	4,2	4,3	4,4	4,5
5,1	5,2	5,3	5,4	5,5

HARD_2.lp

1,1	1,2	1,3	1,4	1,5
2,1	2,2	2,3	2,4	2,5
3,1	3,2	3,3	3,4	3,5
4,1	4,2	4,3	4,4	4,5
5,1	5,2	5,3	5,4	5,5

HARD_3.lp

1,1	1,2	1,3	1,4	1,5
2,1	2,2	2,3	2,4	2,5
3,1	3,2	3,3	3,4	3,5
4,1	4,2	4,3	4,4	4,5
5,1	5,2	5,3	5,4	5,5

If we take a closer look at `hard_input_3.lp` we see that the program requires just 60 milliseconds to find a valid solution that maximizes the number of used tiles (as showed below).

The same input that would require in Minizinc more than 50 minutes is now resolvable in less than one second.

```
input_data_eng > ≡ hard_input_3.lp
1 % ##### Input 10 #####
2
3 % Dimensione griglia : 5
4
5 #const n = 5.
6 #const input_tiles_number = 5.
7 % tile 1 : verde
8 % tile 2 : fuxia
9 % tile 3 : fuxia
10 % tile 4 : viola
11 % tile 5 : azzurro
12
13 tile(1, 1, 1, 0, 0).
14 tile(1, 2, 2, 0, 1).
15 tile(1, 3, 1, 0, 2).
16
17 tile(2, 1, 1, 0, 0).
18 tile(2, 2, 0, 0, 1).
19 tile(2, 3, 3, 1, 0).
20
21 tile(3, 1, 0, 0, 0).
22 tile(3, 2, 1, 0, 1).
23 tile(3, 3, 6, 1, 0).
24
25 tile(4, 1, 1, 0, 0).
26 tile(4, 2, 2, 0, 1).
27 tile(4, 3, 1, 1, 0).
28 tile(4, 4, 4, 1, 1).
29
30 tile(5, 1, 1, 0, 1).
31 tile(5, 2, 2, 1, 0).
32 tile(5, 3, 1, 1, 1).
```

```
Anaconda Prompt (miniconda3)
(potassco) D:\Specialistica\Automated Reasoning\Progetto Esame\Programmi ASP\input_data_eng>clingo completed_code
.lp hard_input_3.lp
clingo version 5.7.1
Reading from completed_code.lp ...
Solving...
Answer: 1
Optimization: 0
Answer: 2
use_tile(3) positioned(3,1,2,1) positioned(3,3,3,1) positioned(3,2,2,2)
Optimization: -1
Answer: 3
use_tile(3) use_tile(4) positioned(3,1,3,2) positioned(3,3,4,2) positioned(4,1,1,3) positioned(4,3,2,3) positione
d(3,2,3,3) positioned(4,2,1,4) positioned(4,4,2,4)
Optimization: -2
Answer: 4
use_tile(1) use_tile(3) use_tile(5) positioned(3,1,4,2) positioned(3,3,5,2) positioned(1,1,3,3) positioned(3,2,4,
3) positioned(5,2,2,4) positioned(1,2,3,4) positioned(5,1,1,5) positioned(5,3,2,5) positioned(1,3,3,5)
Optimization: -3
Answer: 5
use_tile(1) use_tile(3) use_tile(4) use_tile(5) positioned(3,1,2,1) positioned(3,3,3,1) positioned(1,1,5,1) posit
ioned(3,2,2,2) positioned(5,2,4,2) positioned(1,2,5,2) positioned(4,1,1,3) positioned(4,3,2,3) positioned(5,1,3,3
) positioned(5,3,4,3) positioned(1,3,5,3) positioned(4,2,1,4) positioned(4,4,2,4)
Optimization: -4
Answer: 6
use_tile(1) use_tile(2) use_tile(3) use_tile(4) use_tile(5) positioned(3,1,1,1) positioned(3,3,2,1) positioned(4,
1,4,1) positioned(4,3,5,1) positioned(3,2,1,2) positioned(5,2,3,2) positioned(4,2,4,2) positioned(4,4,5,2) positio
ned(1,1,1,3) positioned(5,1,2,3) positioned(5,3,3,3) positioned(1,2,1,4) positioned(2,1,3,4) positioned(2,3,4,4)
positioned(1,3,1,5) positioned(2,2,3,5)
Optimization: -5
OPTIMUM FOUND
Models      : 6
Optimum     : yes
Optimization : -5
Calls       : 1
Time        : 0.060s (Solving: 0.02s 1st Model: 0.00s Unsat: 0.01s)
CPU Time    : 0.047s
```

There is also another input file called “`impossible_input.lp`” that has an input really much bigger compared to the previous one.

If we use the solver with a 7×7 matrix and 13 tiles in input, we obtain:

```
Answer: 8
use_tile(1) use_tile(2) use_tile(3) use_tile(4) use_tile(5) use_tile(8) use_tile(9) use_tile(10) use_tile(11)
use_tile(12) use_tile(13) positioned(13,1,1,1) positioned(13,2,2,1) positioned(13,3,3,1) positioned(11,1,4,1)
positioned(11,2,5,1) positioned(11,3,6,1) positioned(5,2,2,2) positioned(8,1,6,2) positioned(8,2,7,2) position
ed(5,1,1,3) positioned(5,3,2,3) positioned(3,1,4,3) positioned(3,3,5,3) positioned(8,3,7,3) positioned(1,1,1,4
) positioned(4,1,2,4) positioned(4,3,3,4) positioned(3,2,4,4) positioned(2,1,6,4) positioned(2,3,7,4) position
ed(1,2,1,5) positioned(4,2,2,5) positioned(4,4,3,5) positioned(9,2,5,5) positioned(2,2,6,5) positioned(1,3,1,6
) positioned(9,1,4,6) positioned(9,3,5,6) positioned(10,1,1,7) positioned(10,2,2,7) positioned(10,3,3,7) posit
ioned(12,1,5,7) positioned(12,2,6,7) positioned(12,3,7,7)
Optimization: -11
OPTIMUM FOUND

Models      : 8
Optimum     : yes
Optimization : -11
Calls       : 1
Time        : 1155.365s (Solving: 1154.95s 1st Model: 0.02s Unsat: 1154.11s)
CPU Time    : 1152.031s
```

The execution time is also way longer than in previous examples, around 19 minutes.

If we consider the timeout time, the program can place correctly on the grid only 9/13 tiles.

5 Conclusions

We can clearly observe that using Clingo we can solve the problem in a much faster way respect to Minizinc.

Edoardo Diana